

WW-AG · CaptainAgent

Task. In the 2018 VSCode blog post on replit.com, what was the command they clicked on in the last video to remove extra lines?

Gold answer. Format Document.

Annotation. T1, VideoContentAnalysis_Expert: "The code is incorrect for the task."

Prediction. Base score T2; SOAP T1.

T0 VSCode_Expert [12.32 → 19.94] Restates the manager's brief: locate the 2018 post, watch its last video, and report the exact command that removes extra lines.

T1 VideoContentAnalysis_Expert [17.44 → 30.42] **decisive SOAP top-1** Decides to search first and submits code: `results = perform_web_search(query, count=1)` followed by `for result in results: print(result)`. The helper prints its hits itself and returns None, so the loop is bound to fail, and nothing in the plan opens the post or the video.

T2 Computer_terminal [21.64 → 21.64] **base top-1** `exitcode: 1 with TypeError: 'NoneType' object is not iterable`. The helper's own printout still shows the right post, "Zero Setup VSCode Intelligence" (blog.replit.com/intel, 1 July 2018).

T3 VideoContentAnalysis_Expert [15.76 → 28.63] Takes the hit as the post. Cannot view web content, so asks the group to open the link, find the last video, and note the command.

T4 VideoContentAnalysis_Expert [21.45 → 21.45] Repeats the same four instructions and asks for the command to be reported back.

T5 VideoContentAnalysis_Expert [4.87 → 4.87] TERMINATE. No answer is given.

Ranking. Base: T2 > T4 > T1. Rescored: T1 > T3 > T2. The failed run (T2) and the retry (T4) are the visible symptoms; the correction lifts the code that produced them.

WW-HC · Magentic-One

Task. What is the cheapest option to mail a DVD to Colombia from Hartford, Connecticut using FedEx, DHL, or USPS? (Answer as JSON with keys `sender` and `price (usd)`.) **Gold answer.** USPS, 41.75 USD.

Annotation. T8, WebSurfer: "did not return the useful information needed to answer the question." **Prediction.** Base score T73; SOAP T8.

Reading. T0 is the unscored user question. (thought) marks the orchestrator's ledger to itself, → its instruction to an agent. A *Ta-b* entry collapses one ledger / instruction / next-speaker cycle and shows the group's largest score.

T1 Orchestrator (thought) [1.74 → 1.88] Initial plan: fact sheet (a DVD, Hartford to Colombia, three carriers); rates to look up per carrier; team Assistant, ComputerTerminal, FileSurfer, WebSurfer.

T2 Orchestrator (thought) [1.55 → 1.69] Ledger: nothing gathered; start with FedEx. **T3** Orchestrator → WebSurfer [1.49 → 1.49] Look up FedEx rates.

T4 WebSurfer [1.71 → 1.90] Types the query into Bing; returns a screenshot and page metadata.

T5 Orchestrator (thought) [1.82 → 2.00] Ledger: no rates yet; progress. **T6-7** Orchestrator [1.57 → 1.57] Instruction: DHL rates; next speaker.

T8 WebSurfer [2.59 → 2.74] **decisive SOAP top-1** Ignores the DHL instruction; clicks "Calculate Shipping Rates - FedEx". Returns the FedEx rate-tool landing page: screenshot, schema.org metadata, an empty form, no rate.

T9 Orchestrator (thought) [1.82 → 1.95] Ledger: "we have gathered information for FedEx"; DHL and USPS pending. No FedEx rate was ever read; every later turn builds on this belief. **T10-11** Orchestrator [1.56 → 1.56] Instruction: DHL; next speaker.

T12 Orchestrator (thought) [1.51 → 1.68] Ledger: in a loop, errors on FedEx and DHL, no progress; hand to Assistant. **T13-15** Orchestrator → Assistant; Assistant [1.59 → 1.71] Told to get USPS rates, then fix FedEx and DHL; writes a two-step plan ("I'll visit the USPS website") but has no browser; returns nothing.

T16-18 Orchestrator [1.69 → 1.82] Ledger; instruction: USPS; next speaker. **T19** WebSurfer [1.44 → 1.61] Still on the FedEx tool: types "Hartford, Connecticut" into *From*.

T20-22 Orchestrator [1.63 → 1.78] Ledger: "continue with USPS"; next. **T23** WebSurfer [1.62 → 1.76] Clicks the "Hartford, Connecticut, USA" suggestion.

T24-26 Orchestrator [1.57 → 1.71] Ledger; USPS, then revisit DHL; next. **T27** WebSurfer [1.43 → 1.60] Types "Colombia" into the FedEx *To* field.

T28-30 Orchestrator [1.78 → 1.93] Ledger: "continue with USPS"; next. **T31** WebSurfer [1.65 → 1.82] Clicks "Colombia".

T32-34 Orchestrator [1.79 → 1.93] Ledger; USPS; next. **T35** WebSurfer [1.76 → 1.76] Clicks the browser back button; the FedEx form reads *From* Hartford 06103, *destination* Colombia; no rate.

T36-39 Orchestrator (thought) [1.80 → 1.96] Ledger: "Stalled... Re-planning"; new plan: the USPS International Shipping Calculator; ledger. **T40-41** Orchestrator [1.79 → 1.79] Instruction: open the USPS calculator, enter the DVD details; next. **T42** WebSurfer [1.87 → 2.02] Bing search "USPS International Shipping Calculator"; results-page metadata.

T43-45 Orchestrator [1.72 → 1.86] Ledger; click the calculator link, enter the details; next. **T46** WebSurfer [1.52 → 1.71] Clicks "Calculate a Price": the USPS Retail Postage Price Calculator opens, with the destination defaulting to United States.

T47-49 Orchestrator [2.00 → 2.18] Ledger: loop, but on the right page; instruction: ZIP 06101, Colombia, 7.5 × 5.3 × 0.6 in, 0.2 lb; next. **T50** WebSurfer [1.53 → 1.72] Clicks "International".

T51-53 Orchestrator [1.80 → 1.97] Same instruction. **T54** WebSurfer [2.04 → 2.04] Clicks "International" again.

T55-57 Orchestrator [1.81 → 1.99] Same instruction. **T58** WebSurfer [2.24 → 2.46] Types 06101 into the *from*-ZIP field; destination still United States.

T59-61 Orchestrator [1.98 → 2.16] Select Colombia, enter dimensions and weight. **T62** WebSurfer [1.72 → 1.89] Clicks the destination-country field.

T63-65 Orchestrator [1.79 → 1.96] Same instruction. **T66** WebSurfer [1.70 → 1.70] Clicks the field again; nothing selected.

T67-69 Orchestrator (thought) [1.83 → 1.83] Ledger: loop, no progress; "Stalled... Re-planning"; new plan. **T70-72** Orchestrator [1.86 → 2.03] Instruction: open the USPS calculator, fill every field; next. **T73** WebSurfer [2.65 → 2.65] **base top-1** Types "USPS Retail Postage Price Calculator" into Bing once more; returns a 13k-character dump of result metadata. The run ends with no rate from any carrier.

Ranking. Base: T73 > T8 > T58. Rescored: T8 > T73 > T58.

Figure 11: Textual qualitative examples, part 1 of 2: one Who&When trajectory per subset, scored with Qwen3.5-9B under the configuration of Table 6. Each step shows its agent, $[S \rightarrow \hat{S}]$ (base and rescored score, $\times 10^3$), and its content condensed by us; tags mark the annotated decisive step and each score's argmax. In both trajectories the base score peaks on a late symptom and rescoring moves the top-1 onto the annotated step.

TE-Cap · CaptainAgent

Task. I'm researching species that became invasive after people who kept them as pets released them. A certain fish was popularized as a pet by the movie Finding Nemo. According to the USGS, where was this fish found as a non-native species before 2020? Answer as five-digit zip codes, comma-separated.

Gold answer. 34689.

Annotation. T4, InformationExtraction.Expert: "used an inappropriate tool meant for Wikipedia, which failed to extract accurate zip codes from the USGS website."

Prediction. Base score T10; SOAP T4.

T0 Orchestrator [0.76 → 0.84] Assembles the team: InformationExtraction.Expert (Bing search and result parsing), CSVHandling.Expert (Python over local files), Verification.Expert (re-runs checks against cited evidence).

T1 Orchestrator [0.71 → 0.79] Next speaker: InformationExtraction.Expert.

T2 InformationExtraction.Expert [0.98 → 1.07] Identifies the clownfish, *Amphiprion ocellaris*; lays out the plan (find the USGS records, extract the zip codes, format them) and submits `perform_web_search("USGS Amphiprion ocellaris nonnative species locations before 2020", count=5)`.

T3 Orchestrator [0.68 → 0.76] Next speaker: InformationExtraction.Expert.

T4 InformationExtraction.Expert [1.05 → 1.14] **decisive** **SOAP top-1** Picks the USGS Nonindigenous Aquatic Species fact sheet (speciesID 3243) from the hits and calls `scrape_wikipedia_tables(url, header_keyword="States with nonindigenous occurrences")` on it, a helper written for Wikipedia pages.

T5 Orchestrator [0.74 → 0.83] Next speaker: InformationExtraction.Expert.

T6 InformationExtraction.Expert [0.87 → 0.96] Reports "an issue with connecting to the USGS website"; says it will inspect the page "manually" instead, an action no tool in the team supports.

T7 Orchestrator [0.72 → 0.72] Next speaker: InformationExtraction.Expert.

T8 InformationExtraction.Expert [1.03 → 1.13] Claims to have reviewed the page, that it lists locations without zip codes, and "maps" them to 33149, 33139, 33154 (Miami area). ANSWER; TERMINATE.

T9 Orchestrator [0.89 → 1.00] Summary: the zip codes come from a manual review; the scraping failure is noted; "Need to double-check? No."

T10 Orchestrator [1.05 → 1.05] **base top-1** Final ANSWER 33149, 33139, 33154, with the same justification. TERMINATE.

Ranking. Base: T10 > T4 > T8. Rescored: T4 > T8 > T2. The fabricated answer (T8) and its restatement (T10) are the symptoms; the wrong tool call (T4) is the step they were built on.

TE-Mag · Magentic-One

Task. A game-show host hides 30 coins (\$1,000 each) in three boxes; one box holds at least 2 coins and one holds 6 more than another. Bob guesses a count for each box and wins his guess whenever it does not exceed the box, otherwise nothing. With optimal play, what is the minimum amount Bob can win?

Gold answer. 16,000.

Annotation. T6, Orchestrator: "assuming Bob should make the same conservative guess for each box, which ignored optimizing guesses based on varying coin distributions."

Prediction. Base score T9; SOAP T6 (on all three seeds of the triple).

T0 Orchestrator [0.72 → 1.37] Fact sheet: the givens (30 coins, three boxes, the two placement rules, the payoff rule); nothing to look up.

T1 Orchestrator [0.61 → 1.22] Plan: enumerate the valid distributions, derive the guess that maximizes the worst case, implement it with Coder, report.

T2 Coder [0.55 → 1.09] Ledger: proceed to the distributions; not in a loop; progress is being made.

T3 Orchestrator [0.75 → 1.44] Writes `find_valid_distributions()`: loops over (a, b, c) summing to 30 with $a \geq 2$ and some pair differing by 6; prints the list.

T4 Coder [0.50 → 1.05] Ledger: run the script.

T5 Coder [0.52 → 1.10] Ledger: distributions obtained; next, the strategy code.

T6 Orchestrator [0.70 → 1.44] **decisive** **SOAP top-1** Writes `calculate_min_winnings()`: for every distribution sets all three guesses to $\min(a, b, c)$ "as a conservative strategy", sums them, and takes the minimum over distributions. The guesses ignore that Bob chooses per box, so the answer collapses to three times the smallest box.

T7 Coder [0.48 → 1.04] Ledger: run the script.

T8 Coder [0.56 → 1.15] Ledger: execution failed, `NameError: valid_distributions is undefined`; carry the earlier result over; progress: false.

T9 Orchestrator [0.89 → 1.41] **base top-1** Explains the `NameError` and re-issues both functions as one combined script; the strategy is unchanged.

T10 Coder [0.52 → 1.08] Ledger: execute the combined script.

T11 Coder [0.56 → 1.27] Ledger: "Bob can ensure winning 3 coins"; provide the answer.

T12 Coder [0.71 → 0.71] ANSWER \$3,000: the conservative guess always wins at least 3 coins. TERMINATE.

Ranking. Base: T9 > T3 > T0. Rescored: T6 > T3 > T9. The crash report (T8) and the fix (T9) are the symptoms of the strategy fixed in T6.

Figure 12: Textual qualitative examples, part 2 of 2: one TraceElephant trajectory per subset, presented as in Figure 11. On TE-Cap the base score peaks on the final answer and rescoring moves the top-1 six steps earlier, onto the wrong tool call; on TE-Mag it peaks on a bug fix and rescoring moves it three steps earlier, onto the flawed strategy the fix preserved.